

Тестовое задание — Telegram Bot (TRON)

Цель

Не требуется идеальное решение или глубокие знания блокчейна.
Основная цель задания — посмотреть, как ты:

- разбираешься с новой технической областью;
 - ищешь информацию;
 - работаешь с документацией;
 - решаешь проблемы;
 - организуешь код и процесс разработки.
-

Задание

Необходимо реализовать Telegram-бота, который:

1. Принимает TRON-адрес.
 2. Показывает:
 - баланс TRX;
 - баланс USDT (TRC-20);
 - последние транзакции адреса.
 3. Корректно обрабатывает ошибки:
 - невалидный адрес;
 - отсутствие транзакций;
 - недоступность API;
 - прочие нештатные ситуации.
 4. Запускается локально через README-инструкцию.
-

Требования

GitHub Repository

Необходимо:

- создать публичный GitHub-репозиторий и скинуть ссылку сразу по созданию;
- регулярно делать коммиты по мере работы;
- не загружать весь проект одним коммитом в конце.

Будет оцениваться сам процесс работы, а не только финальный результат.

Самостоятельность

Код необходимо писать самостоятельно.

Разрешается:

- читать документацию;
- искать информацию;
- задавать вопросы нейросетям;
- использовать ChatGPT / Claude / Gemini / Cursor как помощников для поиска информации и объяснений.

Не разрешается:

- полностью генерировать проект нейросетями;
- копировать готовое решение без понимания;

Важно понимать и уметь объяснить:

- как работает твой код;
 - почему были приняты те или иные решения;
 - какие проблемы возникали по пути.
-

README

В репозитории должен быть README.md, содержащий:

- инструкцию по запуску;
 - используемые технологии и библиотеки;
 - краткое описание архитектуры;
 - известные ограничения или проблемы, если они есть.
-

История работы

Дополнительно необходимо создать файл WORKLOG.md.

В нём кратко фиксировать:

- какие проблемы возникали;
- как ты их решал;
- что пришлось изучить;
- какие гипотезы не сработали.

Не нужен большой текст — достаточно кратких заметок по ходу работы.

Технические ограничения

Можно использовать любой язык и стек.

Примеры:

- Python
- Node.js / TypeScript
- Go
- и т.д.

Можно использовать:

- публичные API;
 - RPC;
 - SDK;
 - Docker (не обязательно, но будет плюсом).
-

Что будет оцениваться

В первую очередь:

- способность самостоятельно разбираться;
- аккуратность;
- структура проекта;
- умение доводить задачу до результата;

- качество вопросов и решений;
- способность работать с неизвестной областью.

Наличие ошибок не является критичным, если видно понимание процесса и попытки разобраться.

Срок

Ожидаемое время выполнения: 2-4 вечера.

Если что-то не успеваешь реализовать полностью — лучше честно описать текущее состояние и проблемы, чем пытаться скрыть их.